École Normale Supérieure de Lyon
Computer Science Department

Research Internship Report

Set-theoretic types for Ruby

Student:
Zoé Garreau

Supervised by:
Giuseppe Castagna

01/06/2026 – 24/07/2026

Table of contents

1. Introduction 2
2. Background 2
   2.1. Ruby & RBS 3
   2.2. Set-theoretic types & semantic subtyping 4
   2.3. MLsem 4
3. Encoding Ruby in MLsem 5
   3.1. Formalization of the problem 5
   3.2. Variable scoping and the monadic encoding 7
      3.2.1. The heterogeneous state monad 7
      3.2.2. Translating expressions 9
   3.3. Encoding classes and inheritance 10
      3.3.1. Translating classes at the type level 10
      3.3.2. Encoding classes in MLsem expressions 12
      3.3.3. Encoding methods 13
   3.4. Technical details of the full translation 14
4. Prototype 15
5. Conclusion 16
Bibliography 16
A. Full translation 18
   A.1. Featherweight Ruby & RBS 18
   A.2. Monadic encoding 18
   A.3. Ruby Translation 19
   A.4. RBS Translation 20
B. Contexte du stage 21

1. Introduction

Ruby [1] is a dynamically-typed programming language. Although there are several static type checkers for it, they are unsound, meaning they allow programs that produce a type error at runtime. Meanwhile, the theory of set-theoretic types and semantic subtyping is successfully being used in other dynamic languages such as Elixir and Python. In this work we specify and prototype the framework for a new type checker for Ruby based on semantic subtyping. We translate a fragment of Ruby and RBS (Ruby signature language [2]) to MLsem code to harness its implementation of set-theoretic types. This encoding allows us to precisely type Ruby code using union, intersection, negation types and control flow-sensitive informations.

2. Background

Lately, efforts to type dynamic languages have seen sucessful developments. One of the most well-known example is TypeScript [3], a superset of Javascript with static typing. Python has an official type checker called mypy [4] and supports type annotations meant to be used by external type checkers. Elixir is slowly integrating static typing into its compiler [5]. All of these type checkers have been widely adopted by their specific user bases.

A critical property of a type system is soundness, which ensures that the types assigned by the type system correspond to the expected runtime values, and therefore that operations don't fail at runtime because they provided unexpected values.

2.1. Ruby & RBS

Ruby is a dynamically typed and object-oriented programming language. Its main features are classes, modules, mixins and advanced runtime reflection. The base language has no support for static typing or even type annotation syntax, but several external static type checkers exist and have seen widespread use within the Ruby community foremost: Steep [6] and Sorbet [7].

Steep is structured around **R**u**B**y **S**ignature files (RBS) [2], Ruby's official format for type signatures: it type checks Ruby code against one or several RBS files. Steep is a nominal type system, featuring gradual typing, union and intersection types, classes with inheritance and method overloading, and structural interfaces. We will be mainly talking about Steep since Sorbet has similar features, the biggest difference being that it uses inline type annotations instead of RBS files.

because they always treat subclasses as subtypes. This issuThe main issue of these type systems is that they are not sound e was thoroughly studied in [8]. For instance, the following code (using inline signatures and excluding getters for conciseness) is not sound:

```ruby
class Point
  #: (Integer, Integer) -> void
  def initialize(x, y)
    @x = x
    @y = y
  end
  #: (self) -> bool
  def equal(other)
    @x == other.x && @y == other.y
  end
end

class ColorPoint < Point
  #: (Integer, Integer, String) -> void
  def initialize(x, y, color)
    super(x, y)
    @color = color
  end
  #: (self) -> bool
  def equal(other)
    @x == other.x && @y == other.y && @color == other.color
  end
end
```

Figure 1: Unsound example in Steep

In this example we defined two classes, `Point` and `ColorPoint`, the latter inheriting from the former. The `initialize` method is the constructor called when an instance is created, for instance when calling `Point.new(12, 67)`. The `new` class method of a class takes the same arguments as the constructor and returns an instance of the class. `@x` represents an instance variable (or *attribute*).

Given some `p1: ColorPoint` and `p2: Point`, since `ColorPoint` is a subtype of `Point`, we can cast `p1` to a `p: Point`. Then the method call `p.equal(p2)` type checks, but `ColorPoint#equal` is called with a `Point` argument while it expects a `ColorPoint` instance.

2.2. Set-theoretic types & semantic subtyping

Set-theoretic typing models types as sets of values, which can be combined using unions, intersections and negations. These type connectors can type very precisely common idioms found in dynamic languages, such as type cases, pattern matching (using union types `t | u`), function overloading (using intersection types `t & u`)

A basic set-theoretic type system has a few essential constructs, apart from set operations: the top type `any` and bottom type `empty`, tuple types $(t_1, \ldots, t_n)$ which behave as expected, and finally arrow types `t -> u`. Given two types `t` and `u`, `t -> u` is the type of all functions that, given an argument of type `t`, returns only values of type `u`. In particular it makes no claim about what happens in case the function is passed an argument outside of `t` (i.e. in `~t`). There are also singleton types, which contain exactly one value, such as `42`, `true`, etc.

`int | bool` is the union of types `int` and `bool`, i.e. the type of values in either `int` or `bool`. `true | false` is equivalent to `bool`. `(int -> string) & (string -> int)` is the type of functions that accept both arguments of type `int` or `string`, and returns a `string` if the argument was an `int` and vice versa. The domains may not be disjoint: let's say we have a function of type `(a -> c) & (b -> d)` (where `a`, `b`, `c` and `d` are some fixed types). Then passing an argument in `a & b` (in both `a` and `b`) returns a value of type `c & d`. One final example: a function that checks whether a value is a `bool` typically has type `(bool -> true) & (~bool -> false)`.

For a more in-depth introduction to set-theoretic types, see [9].

Semantic subtyping defines the meaning of these set-theoretic connectors in terms of a subtyping relation. This allows the connectors to behave in a more natural way for the programmer than a purely syntactical approach [10].

2.3. MLsem

MLSem [11] is an experimental research language using OCaml-like syntax and semantic subtyping. It is the base implementation of a type checker for the current research on semantic subtyping. It is only used for testing and research purposes.

The syntax of expressions is close to OCaml. Two new features are worth mentioning. The first one is the type-coercion expression `e :> t`, which coerces an expression of type `t'` to some supertype `t` of `t'`, for instance `42 <: (int | bool)`. The second one is the type case expression `if e1 is t then e2 else e3`, which tests whether `e1` is of type `t` and branches accordingly.

The syntax of types is an extension of the one we introduced in the last section, however the MLsem type system supports many more features. We will focus on constructors and records.

Constructors are structural abstract labels that start with a capital letter, akin to polymorphic variants in OCaml, atoms in Elixir or symbols in Ruby. They can also carry data as an optional argument. They have their corresponding types: `Hello : Hello` (the second `Hello` is a singleton type) and `MyInt(67) : MyInt(int)`.

MLsem also supports records. At the expression level, they are exactly like OCaml: a literal syntax `{x = 42; y = "perl"}`, a record update expression `{r with y = true}` and field access `r.x`. At the type level, records are structural: they don't need to be declared. Record types have several forms: closed records `{x : int; y : string}` the type of records that have the *exact* set of specified keys, open records `{x : int; y : string ..}` the type of records that have *at least* the set of specified keys, record update `{t with y : bool}` which updates the type of an existing record type, and finally records with a tail `{x : int; y : string ;; <tail>}` where `<tail>` is either a row variable `` `x `` or a boolean combination of row variables. An intersection tail `` `x & `y `` means that the fields of both `` `x `` and `` `x `` are added, while a union tail means that the fields added are the one in both `` `x `` and `` `y `` (a record type is contravariant in its tail, like its set of keys).

3. Encoding Ruby in MLsem

We type check Ruby code against its corresponding (RBS) signature file by using the existing MLsem type checker: first we translate the RBS and Ruby files into MLsem type definitions and expressions respectively, then we pass the generated code to the MLsem type checker.

3.1. Formalization of the problem

We formalize a fragment of both Ruby [12] and RBS [13]:

Expression	E	::= L | X | C | `self` | `nil` | E.f(E$_1$, ..., E$_n$) | X = E
		| `if` E; E `else` E `end` | E;E | `return` E
Literal	L	::= s | n | `true` | `false`
Statement	S	::= `class` C (< C)$^?$ $\overline{\text{K}}$ `end`
Class statement	K	::= `def` f(E$_1$, ..., E$_n$) E `end`
		| `def initialize`(E$_1$, ..., E$_n$) E `end`
		| `def self.`f(E$_1$, ..., E$_n$) E `end`
		| `attr` x
Binder	X	::= x | `@`x

Figure 2: Featherweight Ruby

Type	T ::=	B | L | C | (T | T) | `T % T` | `not T`
Base type	B ::=	`Symbol` | `Integer` | `self` | `nil` | `bot`
Function type	F ::=	$(T_1, ..., T_n)$ `->` T
Declaration	D ::=	`class` C $\overline{\text{M}}$ `end`
		| `class` C `< C` $\overline{\text{M}}$ `end`
		| `class` C `<: C` $\overline{\text{M}}$ `end`
Member	M ::=	`def` f`: N`
		| `def initialize: N`
		| `def self.`f`: N`
		| `@`x`: T`
Method type	N ::=	F_1 `%` ... `%` F_n
		| F_1 `&` ... `&` F_n

where:

- n ranges over integers
- x, f range over variable names
- s ranges over symbol names
- C ranges over class names

Figure 3: Featherweight RBS

Notation. We note T_1 `&` T_2 := `not (not` T_1 `|` `not` T_2`)`.

Expresions of the form `E.`f`(`$E_1, ..., E_n$`)` are method calls (the only kind of call in Ruby) where `E` is the receiver of the method f and $E_1, ..., E_n$ are the arguments. The `def self.`f`: N` statement (and its RBS counterpart) declares a class method, and `@x: T` declares an attribute `x` of type `T`.

We add a few features to Ruby and RBS (boxed in green) or we modify the meaning of existing ones (boxed in blue).

An `attr` x statement inside a class declares an instance variable. They are used to translate classes more easily. They can be inserted by a preprocessing step that either looks at the attributes defined in RBS or collects used instance variables in the Ruby code.

The `%` type is added to distinguish between the use of `|` for union types and for method overloading, only used inside the translation. We call T_1 `%` T_2 an *overload* type. Finally, the `not` type is a negation type intended to be used by the programmer.

The inheritance syntax has a different meaning than in Steep. In our new syntax, `class` C `<` D only implies C is a subclass of D, not that C is a subtype of D, whereas `class` C `<:` D means that C inherits D *and* that C is a subtype. In the case that the complete signature of C does not allow it to be a subtype of D, the type checker rejects it.

We define 2 functions: $[\![\cdot]\!]_{\text{Ruby}}$ which encodes Ruby code as MLsem expressions, and $[\![\cdot]\!]_{\text{RBS}}$ which encodes RBS signatures as MLsem types. These two functions together should encode the semantics of Ruby programs and types. We type check using MLsem the concatenation of the translated RBS signatures and the translated Ruby code.

3.2. Variable scoping and the monadic encoding

Ruby assignments such as `x = 42` are expressions. This means that evaluating a Ruby expression can change the scope it was evaluated in. For instance, the expression `(x = 12) + x` evaluates to `24` in Ruby and should type check. We call *binding* expressions which extend the context they are evaluated in. MLsem has mutable variables through `let mut` expressions that declare mutable variables that can later be assigned to. But such binding expressions cannot be directly encoded by MLsem's `let mut` expressions because their scoping is only local. This is why we use a monad [14] to encode variables and scopes.

3.2.1. The heterogeneous state monad

We recall that a monad is a type family $M : \text{Type} \to \text{Type}$ together with two operations:

$$\begin{aligned}&\text{pure} : a \to M(a)\\&\text{bind} : M(a) \to (a \to M(b)) \to M(b)\end{aligned}$$

satisfying some coherence laws.

Notation. We write $m \gg\!= f$ for $\text{bind}(m, f)$.

Morally, one can think of monads as a way of encoding side effects in a pure environment. One notable monad is the state monad:

Definition. Let S be a type. We define the *state monad* as the type family $\text{state}_S : A \mapsto (S \to (A \times S))$ with $\text{pure}(x) \coloneqq \lambda s.\ (x, s)$ and $m \gg\!= f \coloneqq \lambda s.\ f(m(s))$.

This monad encodes computations that uses some limited but mutable environment. In fact, if we use a record such as `{x : int; y : int}` for the state, we get a scope-like environment we can use to encode expressions like `(x = 42) + y`. One obvious downside is that the set of variables is fixed: variables are always defined and have the same type throughout the whole scope.

To solve this, we introduce the heterogeneous state monad[1]:

Definition. We define the *heterogeneous state monad*:

$$\texttt{hetState}(\Gamma, \Delta, v, r) \coloneqq \{;;\Gamma\} \to [\texttt{V}(v, \{;;\Gamma\ \texttt{\&}\ \Delta\}) \mid \texttt{R}(r)]$$

where Γ, Δ are row variables, v, r are type variables and $\texttt{V}(...), \texttt{R}(...)$ are constructor types.

Let's unpack this definition. The $\{;;\Gamma\}$ part is the type of records whose fields are exactly contained in Γ, and $\{;;\Gamma\ \texttt{\&}\ \Delta\}$ is $\{;;\Gamma\}$ extended by the fields contained in Δ. Dropping the `R` case, we get $\{;;\Gamma\} \to (v, \{;;\Gamma\ \texttt{\&}\ \Delta\})$, which is the original state monad modified to allow evaluation to extend by Δ its state at the type level. This will allow us to precisely encode the scope of variables. For instance, for the expression $(x = 42) + y$, we can translate the assignment $x = 42$ to an expression of type $\{;;\Gamma\} \to (\texttt{int}, \{x : \texttt{int};;\Gamma\})$ and y to an expression of type $\{y : \texttt{int};;\Gamma\} \to (\texttt{int}, \{y : \texttt{int};;\Gamma\})$ (for some value of Γ). The `R` case is to allow encoding of early returns: we will make this more precise once we have defined the *value* and *bind* operations.

[1] Strictly speaking, this is an abuse of terminology since the type family as well as the two operations do not have the correct signatures.

Definition. The operations associated with the heterogeneous state monad are:

$$\begin{aligned}
&\text{value} : v \to \text{hetState}(\Gamma, \{\}, v, \texttt{empty}) \\
&\text{value}(x) \coloneqq \lambda s.\ \texttt{V}(x, s)
\end{aligned}$$

$$\begin{aligned}
\text{bind} : &\ \text{hetState}\ (\Gamma, \Delta, v, r) \\
&\to (v \to \text{hetState}\ (\Gamma\ \&\ \Delta, \Delta', b, r)) \\
&\to \text{hetState}\ (\Gamma, \Delta\ \&\ \Delta', b, r) \\
\text{bind}(m, f) \coloneqq &\ \lambda\Gamma.\ \text{match}\ m(\Gamma)\ \text{with} \\
&\mid \texttt{V}(v, \Gamma') \to f(v)(\Gamma') \\
&\mid \texttt{R}(r) \to \texttt{R}(r) \\
&\text{end}
\end{aligned}$$

The *value* operation corresponds to the *pure* operation of the state monad: it returns a value without accessing or modifying its scope. The *bind* operation composes two monadic computations with compatible scopes. If both m and $f(v)$ evaluate to a `V`, we get the following chain of scopes:

$$\{;;\Gamma\} \xrightarrow{m} \{;;\Gamma\ \&\ \Delta\} \xrightarrow{f(v)} \{;;\Gamma\ \&\ \Delta\ \&\ \Delta'\}$$

$$\{;;\Gamma\} \xrightarrow{\text{bind}(m,f)} \{;;\Gamma\ \&\ \Delta\ \&\ \Delta'\}$$

If m is an `R`, *bind* will simply ignore the computation of f. This is to model early returns. In the following expression:

```
return 67; puts(12)
```

the computation is short-circuited by `return` and `puts` is never called. We use `R` as a value that bypasses any remaining computations.

Definition. We introduce a new operation for returning[2]:

$$\begin{aligned}
&\text{return} : r \to \text{hetState}(\Gamma, \{\}, \texttt{empty}, r) \\
&\text{return}(x) \coloneqq \lambda s.\ \texttt{R}(x)
\end{aligned}$$

Finally, we introduce operations for manipulating variables:

Definition. Let `x` be a Ruby variable.
We define an operation for accessing `x`:

$$\begin{aligned}
&\text{get}\ \texttt{x} : \text{hetState}\ (\{x : \alpha, ..\}, \{\}, \alpha, \texttt{empty}) \\
&\text{get}\ \texttt{x} = \lambda s.\ \texttt{V}(s.\texttt{x}, s)
\end{aligned}$$

As well as one for assigning to `x`:

$$\begin{aligned}
&\text{set}\ \texttt{x} : \alpha \to \text{hetState}\ (\Gamma, \{x : \alpha\}, \alpha, \texttt{empty}) \\
&\text{set}\ \texttt{x}\ v = \lambda s.\ \texttt{V}(v, \{s\ \text{with}\ \texttt{x} = v\}])
\end{aligned}$$

[2]This is different from the `return` function found in Haskell.

The types of these operations are precise enough to couple types with control flow: when typing branches (such as an `if` or a `case` expression), you get union types:

$$\begin{aligned}
&\texttt{hetState}(\Gamma, \Delta_1, v_1, r_1) \mid \texttt{hetState}(\Gamma, \Delta_2, v_2, r_2) \\
&\quad = (\{;;\Gamma\} \to [\texttt{V}(v_1, \{;;\Gamma \ \&\ \Delta_1\}) \mid \texttt{R}(r_1)]) \mid (\{;;\Gamma\} \to [\texttt{V}(v_2, \{;;\Gamma \ \&\ \Delta_2\}) \mid \texttt{R}(r_2)]) \\
&\quad <: \{;;\Gamma\} \to [\texttt{V}(v_1, \{;;\Gamma \ \&\ \Delta_1\}) \mid \texttt{V}(v_2, \{;;\Gamma \ \&\ \Delta_2\}) \mid \texttt{R}(r_1 \mid r_2)] \\
&\quad <: \{;;\Gamma\} \to (\texttt{V}(v_1 \mid v_2, \{;;\Gamma \ \&\ (\Delta_1 \mid \Delta_2)\}) \mid \texttt{R}(r_1 \mid r_2)) \\
&\quad <: \texttt{hetState}(\Gamma, (\Delta_1 \mid \Delta_2), (v_1 \mid v_2), (r_1 \mid r_2))
\end{aligned}$$

Taking the union of the monads is more precise than taking the union of the context and values component-wise. For a concrete example, see Figure 15.

3.2.2. Translating expressions

We can finally define the full translation of expressions. Note that MLsem cannot express first-class key polymorphism nor row variables in type aliases at the time of writing this report, so the definition of hetState as well as *value*, *return*, *bind*, *get* and *set* need to be expanded when used in the translation.

Notation. We note $\texttt{truthy} := \texttt{~(false | ())}$.

$$\begin{aligned}
[\![\texttt{L}]\!]_{\texttt{E}} &= \text{value } \texttt{L} \\
[\![\texttt{x}]\!]_{\texttt{E}} &= \text{get } \texttt{x} \\
[\![\texttt{@x}]\!]_{\texttt{E}} &= \text{value self._attr_x} \\
[\![C]\!]_{\texttt{E}} &= \text{value } C \\
[\![\texttt{nil}]\!]_{\texttt{E}} &= \text{value } \texttt{()} \\
[\![\texttt{self}]\!]_{\texttt{E}} &= \text{value } \texttt{self} \\
[\![\texttt{E.f(}\texttt{E}_1, ..., \texttt{E}_n\texttt{)}]\!]_{\texttt{E}} &= [\![\texttt{E}]\!] \gg\!= \lambda r.\ [\![\texttt{E}_1]\!] \gg\!= \lambda a_1. \cdots [\![\texttt{E}_n]\!] \gg\!= \lambda a_n.\ \text{value } r.\texttt{f}(a_1, ..., a_n) \\
[\![\texttt{return E}]\!]_{\texttt{E}} &= [\![\texttt{E}]\!] \gg\!= \text{return} \\
[\![\texttt{x = E}]\!]_{\texttt{E}} &= [\![\texttt{E}]\!] \gg\!= (\text{set } \texttt{x}) \\
[\![\texttt{@x = E}]\!]_{\texttt{E}} &= [\![\texttt{E}]\!] \gg\!= \lambda v.(\text{self} := \{\text{self with self._attr_x} = v\}; \text{value } v) \\
[\![\texttt{if E}_1\texttt{; E}_2\texttt{ else E}_3\texttt{ end}]\!]_{\texttt{E}} &= [\![\texttt{E}_1]\!] \gg\!= \lambda b.\ \text{ if } b \text{ is truthy then } [\![\texttt{E}_2]\!] \text{ else } [\![\texttt{E}_3]\!] \\
[\![\texttt{E}_1\texttt{ ; E}_2]\!]_{\texttt{E}} &= [\![\texttt{E}_1]\!] \gg\!= \lambda_.\ [\![\texttt{E}_2]\!]
\end{aligned}$$

Figure 4: Translation of expressions

Some examples of translations:

$$\begin{aligned}
\texttt{(x = 1); x} \ \rightsquigarrow\ & \text{value } 1 \\
& \gg\!= \lambda v.\ \text{set x } v \\
& \gg\!= \lambda_.\ \text{get x}
\end{aligned}$$

$$\begin{aligned}
\texttt{return self.set_level(10)} \ \rightsquigarrow\ & \text{value self} \\
& \gg\!= \lambda r.\ \text{value } 10 \\
& \gg\!= \lambda a.\ \text{value } r.\text{set_level}(a) \\
& \gg\!= \text{return}
\end{aligned}$$

Ruby expressions are translated into fairly standard monadic code using the operations we defined. We translate `if` expressions to check for `truthy` values instead of just `true` because the only falsy values in Ruby are `false` and `nil`. Finally, the translation of instance variables (`@x` and `@x = E`) will be explained alongside the encoding of classes in the next section.

3.3. Encoding classes and inheritance

One of Ruby's main features is its classes, which we cannot translate directly in MLsem. Classes have several features we want to encode in the type system: attributes, methods, inheritance, the `self` type, nominal typing, and being able to control whether inheritance generates a subtype.

We suppose that we have two values `rec : ('a -> 'a) -> 'a` to encode recursion at the type level and `opaque : 'a` to get opaque values of any type (using type casts: `opaque :> int`). They can be declared by top-level statements in MLsem:

```
val rec : ('a -> 'a) -> 'a
val opaque : 'a
```

3.3.1. Translating classes at the type level

We need to encode classes as runtime values in MLsem, since classes are first-class values in Ruby. We encode classes and instances as recursive records, whose fields contain attributes and methods. Nominal subtyping is encoded by a special extra field.

The translation of classes is composed of two parts: the RBS declaration is translated to MLsem types and the Ruby code is translated into an MLsem expression.

Let's take a simple class signature as an example:

```
class IntWrapper
  @x: Integer

  def initialize: (Integer) -> top
  def get_x: () -> Integer
  def equal: (self) -> bool
end
```

Figure 5: `IntWrapper` RBS class signature

At the type level, a class generates three types: the types of its instances and the type of the class singleton. In Figure 6, the `tyClassIntWrapper` type represents the type of the class singleton. The fields of the record (except for `__name`) represent the class methods of the class. In the case of `IntWrapper`, the only one is `new`, whose type was generated from the signature of `initialize`. The type of `IntWrapper` instances is defined by open recursion: in `tyIntWrapperRec`, the `'self` type parameter is bound to represent the `self` type to allow changing when a class inherits from `IntWrapper`. Finally, the type `tyIntWrapper` closes the recursion, instantiating the self type to `tyIntWrapper`.

```
type tyIntWrapperRec('self) = {
  __name : ~Inst__A;
  __attr_x : int;
  get_x : () -> int;
  equal : 'self -> bool
  ..
}
and tyIntWrapper = tyIntWrapperRec(tyIntWrapper)
and tyClassIntWrapper = {
  __name : ~Class__IntWrapper;
  new : (int) -> tyIntWrapper
  ..
}
```

Figure 6: `IntWrapper` MLsem type translation

We give another example to illustrate how inheritance and nominal typing is encoded:

```
class A
  def get_level: () -> Integer
end
class B <: A end
class C <: B end
class D <: A end
```

Figure 7: Inheritance with subtyping example

This example produces the following types (omitting the class types):

```
type tyARec('self) =
  { __name : ~Inst__A; eq : () -> int .. }
type tyA = tyARec(tyA)

type tyBRec('self) =
  { tyARec('self) with __name : ~(Inst__B | Inst__A) }
type tyB = tyBRec(tyB)

type tyCRec('self) =
  { tyBRec('self) with __name : ~(Inst__C | Inst__B | Inst__A) }
type tyC = tyCRec(tyC)

type tyDRec('self) =
  { tyARec('self) with __name : ~(Inst__D | Inst__A) }
type tyD = tyDRec(tyD)
```

Figure 8: Translation of Figure 7

We use the record type update syntax to include the parent's fields into the type. The `__name` field encodes nominal typing, ensuring that two different classes with the same

signatures stay distinct for the type checker. However, we want to allow a subclass to be a subtype of its parent class, so the field must take into account every transtive parents in the inheritance tree (Figure 9). We negate the type to account for variance: we want the `__name` field of a subclass to be a subtype of the `__name` of its parent class. Making the `__name` field contravariant with respect to inheritance mimics the fact that the set of keys in a record is contravariant.

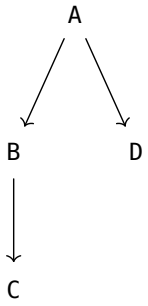

Figure 9: Subtyping tree of Figure 7

Finally, a check is added ensure that the type generated by a `:>` subclass is a subtype:

```
let _checkVal = ((opaque :> tyB) :> tyA)
```

This encoding fixes the issue of subtyping in the presence of inheritance highlighted in [8] because the type checker ensures that a subtype is being generated when the subclass is explicitly specified to be a subtype. Suppose we have the following signature:

```
class A
  def equal: (self) -> bool
end
class B <: A
  def equal: (self) -> bool
end
```

Figure 10: Incorrect use of subtyping inheritance

We get the following types (after simplifying open recursion and record updates):

```
tyA = { __name : ~Inst__A            ; eq : tyA -> bool .. }
tyB = { __name : ~(Inst__A | Inst__B); eq : tyB -> bool .. }
```

Due to `'self` appearing in a contravariant position, `tyB` is not a subtype of `tyA` despite `B` inheriting from `A`, so the subtyping check fails and the signature does not type check, thus restoring soundness. We can however inherit from `A` without generating a subtype: replacing `class B <: A` by `class B < A` yields the following types:

```
tyA = { __name : ~Inst__A; eq : tyA -> bool .. }
tyB = { __name : ~Inst__B; eq : tyB -> bool .. }
```

With `<` inheritance, the `__name` field only refers to the current class.

3.3.2. Encoding classes in MLsem expressions

A class `A` generates a top-level statement of the form (excluding some type coercions for clarity):

```
let classA =
  rec (fun (self : tyClassA) ->
    let mut self = self in
    {
      __name = (opaque :> ~Class__A);
      (* class methods *)
    }
  )
```

Figure 11: Top level statement generated from class `A`

The `new` class method has a particular translation of the form (again, omitting type coercions):

```
new =
  fun args -> rec (fun (self : tyA) ->
    let mut self = self in
    self := {
      __name = (opaque :> ~Inst__A);
      (* instance methods and attributes *)
    };
    (* translation of the initialize method *)
    self
  )
```

Figure 12: The `new` field generated from class `A`

In both the class and instance translations, the `__name` field is present to reflect our type-level encoding. Similarly, an attribute is translated to a special field initialized to `opaque`. Our translation does not keep track of the initialization state of attributes.

3.3.3. Encoding methods

Instance methods and class methods have a non-trivial encoding because of the monadic encoding of expressions. A translation of methods needs to:

- initialize the method's context
- add the method's arguments to the context
- translate the body of the function
- extract the return value from the monad.

Definition. We define an *extract* operation:

$$\begin{aligned}\text{extract}((\mathrm{x}_1, ..., \mathrm{x}_n), \mathtt{E}) := \text{match } [\![\mathtt{E}]\!](\{\mathrm{x}_1 = \mathrm{x}_1; ...; \mathrm{x}_n = \mathrm{x}_n\}) \text{ with} \\ \mid \mathtt{V}(v, \_) \to v \\ \mid \mathtt{R}(r) \to r \\ \text{end}\end{aligned}$$

The *extract* operation takes a list of Ruby variables and an expression.

We can then define the translation of a method as:

```
def my_method(x, y)
  <body>
end
```

⇝

```
my_method =
  fun (x, y) ->
    extract((x, y), <body>)
```

Figure 13: Translation of methods

The fact that the monadic state is only managed by the method means that the monadic encoding is transparent at the type level, so that function types are not polluted with an extra state argument.

RBS allows the programmers to overload methods:

```
def process: (42) -> String
           % (Integer) -> Integer
           % (Symbol) -> Symbol
```

(formalized syntax)

```
def process: (42) -> String
           | (Integer) -> Integer
           | (Symbol) -> Symbol
```

(real RBS syntax)

Figure 14: Method overloading in RBS

Contrary to MLsem intersection types, this overloading mecanism is order-sensitive. If one passes `42` to `process`, the first overload matches the type of the argument passed and the second overload is never considered. This type can be rewritten using intersection types:

```
(42 -> String) & (Integer \ 42 -> Integer) & (Symbol -> Symbol)
```

Definition. We define *domain* and *codomain* functions for RBS types:

$$\mathrm{dom}(\mathtt{T}) = \begin{cases} (\mathtt{U}_1, ..., \mathtt{U}_n) & \text{if } \mathtt{T} = \mathtt{(U_1, ..., U_n) \ -> \ R} \\ \mathrm{dom}(\mathtt{T}_1) \ \mathtt{|} \ \mathrm{dom}(\mathtt{T}_2) & \text{if } \mathtt{T} = \mathtt{T}_1 \ \mathtt{\%} \ \mathtt{T}_2 \\ \text{undefined} & \text{otherwise} \end{cases}$$

$$\mathrm{cod}(\mathtt{T}) = \begin{cases} \mathtt{R} & \text{if } \mathtt{T} = \mathtt{(U_1, ..., U_n) \ -> \ R} \\ \text{undefined} & \text{otherwise} \end{cases}$$

We assume chains of `%` are always of the form $((\mathtt{F}_1 \ \mathtt{\%} \ \mathtt{F}_2) \ \mathtt{\%} \ ...) \ \mathtt{\%} \ \mathtt{F}_n$. We can then define the translation of overload types as:

$$[\![\mathtt{T}_1 \ \mathtt{\%} \ \mathtt{T}_2]\!]_\mathtt{T} := [\![\mathtt{T}_1]\!] \ \mathtt{\&} \ ([\![\mathrm{dom}(\mathtt{T}_2)]\!] \ \mathtt{\setminus} \ [\![\mathrm{dom}(\mathtt{T}_1)]\!] \ \mathtt{->} \ [\![\mathrm{cod}(\mathtt{T}_2)]\!])$$

Since we defined classes with open recursion at the type level, we also define:

$$[\![\mathtt{self}]\!]_\mathtt{T} := \mathtt{'self}$$

The remaining cases of $[\![\cdot]\!]_\mathtt{T}$ are straightforward.

3.4. Technical details of the full translation

The full translation can be found in Appendix A.

To actually implement the translation, you need to insert `attr x` statements in the Ruby code, since you don't declare attributes in Ruby. This can be done simply by collecting every attribute mentioned in the class and then inserting the `attr` statements. The translation also assumes the `initialize` method is defined both in Ruby and RBS in order to generate the `new` class method.

Another pass is needed to build the inheritance hierarchy, to generate the `__name` field (this can be done during the translation of the RBS signatures).

You can then translate the RBS, and then concatenate it with the translation of the Ruby code. You also need to prepend the definition of the several constants we needed during the translation (as shown in Figure 17).

4. Prototype

An implementation of the full translation is available on Github [15]. Several examples can be found in the `test` folder of the repository. The output can be read (the generated MLsem code should be nicely formatted) and pasted directly into MLsem [16].

We give two examples of programs that we are able to type check using our translation and MLsem:

```
1  def f(b)
2    if b then
3      x = 1
4    else
5      y = 2
6    end
7    if b then
8      z = x
9    else
10     z = y
11   end
12   z
13 end
```

Figure 15: Flow-sensitive scopes

This code type checks for both signatures:

```
1 def f: (bool) -> Integer
2 def f: (true) -> 1 | (false) -> 2
```

Our encoding is capable of capturing control flow-based typing information. Trying to access `x` outside of the second `if` expression would result in a type error because `x` is not defined in the case that `b` is falsy, but the encoding is precise enough to analyze the branches. It can also determine that `z` is defined in all cases after the second `if` expression. This sort of flow-based analysis can become very useful if integrated with type cases in the future (using `Object#is_a?` [17]).

This next example involves reading and writing to an attribute:

```
1  class A
2    attr x
3
4    def initialize(x)
5      @x = x
6    end
7
8    def get_x()
9      @x
10   end
11 end
```

```
1 class A
2   @x: Integer
3
4   def initialize: (Integer) -> top
5   def get_x: () -> Integer
6 end
```

Figure 16: Methods and attributes inside a class

Changing `initialize: (Integer) -> top` to `initialize: (Symbol) -> top` makes the example no longer compile, as expected, since the assignment `@x = x` tries to assign a `Symbol` where an `Integer` is expected.

5. Conclusion

We have encoded Ruby programs inside MLsem, allowing us to type Ruby programs using the existing semantic subtyping framework, addressing soundness holes (assuming the translation is indeed correct) by providing new constructs (the two types of inheritance). We implemented a small type checker for Ruby using the translation we defined, which can be used for further testing. The various examples tested using our prototype hint that set-theoretic types and semantic subtyping are a useful approach to type Ruby programs.

The next step is to integrate more features to our encoding such as type variables and gradual typing. Type variables must also be able to quantify over the subclass hierarchy of a particular class using F-bounded polymorphism [18] or the matching relation brought up in [8]: since subclasses are no longer subtypes simple subtyping constraints are not as useful for polymorphism. Gradual typing is also essential to type dynamic languages to allow the programmer to migrate their untyped code progressively instead of require the whole program to be typed. It would be also useful to have a proof that our encoding is sound and preserves operational semantics, as this would increase our confidence in the encoding.

Bibliography

[1] The Ruby Community, "Ruby programming language." [Online]. Available: https://www.ruby-lang.org/en/

[2] The Ruby Community, "RBS." [Online]. Available: https://github.com/ruby/rbs

[3] Microsoft, "TypeScript." [Online]. Available: https://www.typescriptlang.org/

[4] "mypy." [Online]. Available: https://www.mypy-lang.org/index.html

[5] G. Castagna, G. Duboc, and J. Valim, "The Design Principles of the Elixir Type System," *The Art, Science, and Engineering of Programming*, vol. 8, no. 2, 2024.

[6] S. Matsumoto, "Steep." [Online]. Available: https://github.com/soutaro/steep

[7] Stripe, "Sorbet." [Online]. Available: https://sorbet.org/

[8] K. Bruce, L. Cardelli, G. Castagna, The~Hopkins~Object~Group, G. Leavens, and B. Pierce, "On Binary Methods," *Theory and Practice of Object Systems*, vol. 1, no. 3, pp. 221–242, 1996.

[9] G. Castagna, "Programming with Union, Intersection, and Negation Types," in *The French School of Programming*, B. Meyer, Ed., Springer, 2024, pp. 309–378.

[10] A. Frisch, G. Castagna, and V. Benzaken, "Semantic subtyping: Dealing set-theoretically with function, union, intersection, and negation types," *J. ACM*, vol. 55, no. 4, Sep. 2008, doi: 10.1145/1391289.1391293.

[11] G. Castagna, M. Laurent, and K. Nguyễn, "Polymorphic Type Inference for Dynamic Languages," *Proc. ACM Program. Lang.*, vol. 8, no. POPL, Jan. 2024, doi: 10.1145/3632882.

[12] The Ruby Community, "Ruby syntax documention." [Online]. Available: https://docs.ruby-lang.org/en/4.0/syntax_rdoc.html

[13] The Ruby Community, "RBS syntax documention." [Online]. Available: https://github.com/ruby/rbs/blob/master/docs/syntax.md

[14] N. Benton, J. Hughes, and E. Moggi, "Monads and Effects," in *Applied Semantics*, G. Barthe, P. Dybjer, L. Pinto, and J. Saraiva, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2002, pp. 42–122.

[15] Z. Garreau, "RbSem." [Online]. Available: https://github.com/zowepsilon/rbsem

[16] M. Laurent, "MLsem." [Online]. Available: https://e-sh4rk.github.io/MLsem/

[17] The Ruby Community, "Object#is_a? method documentation." [Online]. Available: https://docs.ruby-lang.org/en/4.0/Object.html#method-i-is_a-3F

[18] P. Canning, W. Cook, W. Hill, W. Olthoff, and J. C. Mitchell, "F-bounded polymorphism for object-oriented programming," in *Proceedings of the Fourth International Conference on Functional Programming Languages and Computer Architecture*, in FPCA '89. Imperial College, London, United Kingdom: Association for Computing Machinery, 1989, pp. 273–280. doi: 10.1145/99370.99392.

A. Full translation

A.1. Featherweight Ruby & RBS

Expression $\mathtt{E} ::= \mathtt{L} \mid \mathtt{X} \mid C \mid \mathtt{self} \mid \mathtt{nil} \mid \mathtt{E}.f(\mathtt{E}_1, ..., \mathtt{E}_n) \mid \mathtt{X = E}$
$\mid \mathtt{if\ E;\ E\ else\ E\ end} \mid \mathtt{E;E} \mid \mathtt{return\ E}$

Literal $\mathtt{L} ::= s \mid n \mid \mathtt{true} \mid \mathtt{false}$

Statement $\mathtt{S} ::= \mathtt{class}\ C\ (\mathtt{<}\ C)^?\ \ \overline{\mathtt{K}}\ \mathtt{end}$

Class statement $\mathtt{K} ::= \mathtt{def}\ f(x_1, ..., x_n)\ \mathtt{E\ end}$
$\mid \mathtt{def\ initialize}(x_1, ..., x_n)\ \mathtt{E\ end}$
$\mid \mathtt{def\ self.}f(x_1, ..., x_n)\ \mathtt{E\ end}$
$\mid \boxed{\mathtt{attr}\ x}$

Binder $\mathtt{X} ::= x \mid \mathtt{@}x$

Type $\mathtt{T} ::= \mathtt{B} \mid \mathtt{L} \mid C \mid (\mathtt{T\ |\ T}) \mid \boxed{\mathtt{T\ \%\ T}} \mid \boxed{\mathtt{not\ T}}$

Base type $\mathtt{B} ::= \mathtt{Symbol} \mid \mathtt{Integer} \mid \mathtt{self} \mid \mathtt{nil} \mid \mathtt{bot}$

Function type $\mathtt{F} ::= (\mathtt{T}_1, ..., \mathtt{T}_n)\ \mathtt{->\ T}$

Declaration $\mathtt{D} ::= \mathtt{class}\ C\ \ \overline{\mathtt{M}}\ \mathtt{end}$
$\mid \mathtt{class}\ C\ \boxed{\mathtt{<}\ C}\ \ \overline{\mathtt{M}}\ \mathtt{end}$
$\mid \mathtt{class}\ C\ \boxed{\mathtt{<:}\ C}\ \ \overline{\mathtt{M}}\ \mathtt{end}$

Member $\mathtt{M} ::= \mathtt{def}\ f\mathtt{:\ N}$
$\mid \mathtt{def\ initialize:\ N}$
$\mid \mathtt{def\ self.}f\mathtt{:\ N}$
$\mid \mathtt{@}x\mathtt{:\ T}$

Method type $\mathtt{N} ::= \mathtt{F}_1\ \mathtt{\%}\ ...\ \mathtt{\%}\ \mathtt{F}_n$
$\mid \mathtt{F}_1\ \mathtt{\&}\ ...\ \mathtt{\&}\ \mathtt{F}_n$

where:

- n ranges over integers
- x, f range over variable names
- s ranges over symbol names
- C ranges over class names

Notation. We note $\mathtt{T}_1\ \mathtt{\&}\ \mathtt{T}_2 ::= \mathtt{not}\ (\mathtt{not\ T}_1 \mid \mathtt{not\ T}_2)$.

A.2. Monadic encoding

$$\mathtt{hetState}(\Gamma, \Delta, v, r) ::= \{;; \Gamma\} \to [\mathtt{V}(v, \{;; \Gamma\ \&\ \Delta\}) \mid \mathtt{R}(r)]$$

where Γ, Δ are row variables and v, r are type variables.

$$\text{value} : v \to \text{hetState}(\Gamma, \{\}, v, \mathtt{empty})$$
$$\text{value}(x) ::= \lambda s.\ \mathtt{V}(x, s)$$

$$
\begin{aligned}
\text{bind} : \text{hetState}\ (\Gamma, \Delta, v, r) \\
&\to (v \to \text{hetState}\ (\Gamma\ \&\ \Delta, \Delta', b, r)) \\
&\to \text{hetState}\ (\Gamma, \Delta\ \&\ \Delta', b, r) \\
\text{bind}(m, f) := \lambda\Gamma.\ &\text{match}\ m(\Gamma)\ \text{with} \\
&\mid \mathtt{V}(v, \Gamma') \to f(v)(\Gamma') \\
&\mid \mathtt{R}(r) \to \mathtt{R}(r) \\
&\text{end}
\end{aligned}
$$

$$
\begin{aligned}
\text{return} &: r \to \text{hetState}(\Gamma, \{\}, \mathtt{empty}, r) \\
\text{return}(x) &:= \lambda s.\ \mathtt{R}(x)
\end{aligned}
$$

$$
\begin{aligned}
\text{get}\ \mathtt{x} &: \text{hetState}\ (\{x : \alpha, ..\}, \{\}, \alpha, \mathtt{empty}) \\
\text{get}\ \mathtt{x} &= \lambda s.\ \mathtt{V}(s.\mathtt{x}, s)
\end{aligned}
$$

$$
\begin{aligned}
\text{set}\ \mathtt{x} &: \alpha \to \text{hetState}\ (\Gamma, \{x : \alpha\}, \alpha, \mathtt{empty}) \\
\text{set}\ \mathtt{x}\ v &= \lambda s.\ \mathtt{V}(v, \{s\ \text{with}\ \mathtt{x} = v\}])
\end{aligned}
$$

$$
\begin{aligned}
\text{extract}((\mathrm{x}_1, ..., \mathrm{x}_n), \mathtt{E}) := &\ \text{match}\ [\![\mathtt{E}]\!](\{\mathrm{x}_1 = \mathrm{x}_1; ...; \mathrm{x}_n = \mathrm{x}_n\})\ \text{with} \\
&\mid \mathtt{V}(v, \_) \to v \\
&\mid \mathtt{R}(r) \to r \\
&\text{end}
\end{aligned}
$$

Notation. We write $m \gg\!= f$ for bind m f.

A.3. Ruby Translation

```
type truthy = ~(false | ())
val opaque: 'a
val rec: ('a -> 'a) -> 'a
```

Figure 17: Beginning of a translated file

$$
\begin{aligned}
[\![\mathtt{L}]\!]_{\mathtt{E}} &= \text{value}\ \mathtt{L} \\
[\![\mathtt{x}]\!]_{\mathtt{E}} &= \text{get}\ \mathtt{x} \\
[\![\mathtt{@x}]\!]_{\mathtt{E}} &= \text{value self.\_attr\_x} \\
[\![C]\!]_{\mathtt{E}} &= \text{value}\ C \\
[\![\mathtt{nil}]\!]_{\mathtt{E}} &= \text{value}\ \mathtt{()} \\
[\![\mathtt{self}]\!]_{\mathtt{E}} &= \text{value}\ \mathtt{self} \\
[\![\mathtt{E.f(E}_1, ..., \mathtt{E}_n\mathtt{)}]\!]_{\mathtt{E}} &= [\![\mathtt{E}]\!] \gg\!= \lambda r.\ [\![\mathtt{E}_1]\!] \gg\!= \lambda a_1.\ \cdots\ [\![\mathtt{E}_n]\!] \gg\!= \lambda a_n.\ \text{value}\ r.\mathtt{f}(a_1, ..., a_n) \\
[\![\mathtt{return\ E}]\!]_{\mathtt{E}} &= [\![\mathtt{E}]\!] \gg\!= \text{return} \\
[\![\mathtt{x\ =\ E}]\!]_{\mathtt{E}} &= [\![\mathtt{E}]\!] \gg\!= (\text{set}\ \mathtt{x}) \\
[\![\mathtt{@x\ =\ E}]\!]_{\mathtt{E}} &= [\![\mathtt{E}]\!] \gg\!= \lambda v.(\text{self} := \{\text{self with self.\_attr\_x} = v\}; \text{value}\ v) \\
[\![\mathtt{if\ E}_1\mathtt{;\ E}_2\ \mathtt{else\ E}_3\ \mathtt{end}]\!]_{\mathtt{E}} &= [\![\mathtt{E}_1]\!] \gg\!= \lambda b.\ \ \text{if}\ b\ \text{is truthy then}\ [\![\mathtt{E}_2]\!]\ \text{else}\ [\![\mathtt{E}_3]\!] \\
[\![\mathtt{E}_1\ \mathtt{;\ E}_2]\!]_{\mathtt{E}} &= [\![\mathtt{E}_1]\!] \gg\!= \lambda\_.\ [\![\mathtt{E}_2]\!]
\end{aligned}
$$

$$[\![\texttt{class name ((<|<:) sup))}^{?}\ \texttt{K}_1\texttt{,...,K}_n\ \ \texttt{end}]\!]_{\texttt{S}} =$$

```
let name = rec (fun self ->
  let mut self = self in {
  ((opaque :> supClass)) with)?
  __name = (opaque :> ~ℕ^Class(name));
  ⟦K_1⟧^Class; ...; ⟦K_n⟧^Class
})
```

$$[\![\texttt{def initialize(x}_1,\ldots,\texttt{x}_n\texttt{) E end}]\!]^{\texttt{Class}}_{\texttt{K}} =$$

```
    new = fun (x_1, ..., x_n) -> rec (fun self ->
      let mut self = self in
      self := {
        ((opaque :> sup_(name)) with)?
        __name = (opaque :> ~ℕ^Inst(name));
        ⟦K_1⟧^Inst; ...; ⟦K_n⟧^Inst;
      };
      extract ((x_1, ..., x_n), ⟦E⟧);
      self
    );
```

$$[\![\texttt{def self.}f\texttt{(x}_1,\ldots,\texttt{x}_n\texttt{) E end}]\!]^{\texttt{Class}}_{\texttt{K}} = f\ \texttt{=}\ \texttt{fun}\ (\texttt{x}_1,\ldots,\texttt{x}_n)\ \texttt{->}\ \text{extract}\ ((\texttt{x}_1,\ldots,\texttt{x}_n),[\![\texttt{E}]\!]);$$

$$[\![\texttt{def}\ f\texttt{(x}_1,\ldots,\texttt{x}_n\texttt{) E end}]\!]^{\texttt{Inst}}_{\texttt{K}} = f\ \texttt{=}\ \texttt{fun}\ (\texttt{x}_1,\ldots,\texttt{x}_n)\ \texttt{->}\ \text{extract}\ ((\texttt{x}_1,\ldots,\texttt{x}_n),[\![\texttt{E}]\!]);$$

$$[\![\texttt{attr}\ x]\!]^{\texttt{Inst}}_{\texttt{K}} = \texttt{__attr\_}x\ \texttt{=}\ \texttt{opaque;}$$

A.4. RBS Translation

$$\begin{aligned}
[\![\texttt{Symbol}]\!]_{\texttt{T}} &= \texttt{enum}\\
[\![\texttt{Integer}]\!]_{\texttt{T}} &= \texttt{int}\\
[\![\texttt{self}]\!]_{\texttt{T}} &= \texttt{'self}\\
[\![\texttt{nil}]\!]_{\texttt{T}} &= \texttt{()}\\
[\![\texttt{bot}]\!]_{\texttt{T}} &= \texttt{empty}\\
[\![\texttt{T}_1\ \texttt{|}\ \texttt{T}_2]\!]_{\texttt{T}} &= [\![\texttt{T}_1]\!]\ \texttt{|}\ [\![\texttt{T}_2]\!]\\
[\![\texttt{T}_1\ \texttt{\%}\ \texttt{T}_2]\!]_{\texttt{T}} &= [\![\texttt{T}_1]\!]\ \texttt{\&}\ ([\![\text{dom}(\texttt{T}_2)]\!]\ \texttt{\textbackslash}\ [\![\text{dom}(\texttt{T}_1)]\!]\ \texttt{->}\ [\![\text{cod}(\texttt{T}_2)]\!])\\
[\![C]\!]_{\texttt{T}} &= C\\
[\![\texttt{not T}]\!]_{\texttt{T}} &= \texttt{\~{}}[\![\texttt{T}]\!]\\
[\![\texttt{L}]\!]_{\texttt{T}} &= \texttt{L}\\
[\![\texttt{(T}_1,\ldots,\texttt{T}_n\texttt{) -> U}]\!]_{\texttt{F}} &= ([\![\texttt{T}_1]\!],\ldots,[\![\texttt{T}_n]\!])\ \texttt{->}\ [\![\texttt{U}]\!]
\end{aligned}$$

$$\text{dom}(\texttt{T}) = \begin{cases} \texttt{(U}_1,\ldots,\texttt{U}_n\texttt{)} & \text{if } \texttt{T} = \texttt{(U}_1,\ldots,\texttt{U}_n\texttt{) -> R} \\ \text{dom}(\texttt{T}_1)\ \texttt{|}\ \text{dom}(\texttt{T}_2) & \text{if } \texttt{T} = \texttt{T}_1 \mathbin{⅋} \texttt{T}_2 \\ \text{undefined} & \text{otherwise} \end{cases}$$

$$\text{cod}(\texttt{T}) = \begin{cases} \texttt{R} & \text{if } \texttt{T} = \texttt{(U}_1,\ldots,\texttt{U}_n\texttt{) -> R} \\ \text{undefined} & \text{otherwise} \end{cases}$$

$$\mathbb{N}^{\texttt{Inst}}(C_1) := \texttt{\_C}_1 \texttt{ | ... | \_C}_n$$
where $\{C_i\}$ is the set of parents of C_1 in the subtyping tree
$$\mathbb{N}^{\texttt{Class}}(C_1) := \texttt{\_Class\_C}_1 \texttt{ | \_Class}$$

$[\![\texttt{class name ((<|<:) sup)}^?\ \texttt{M}_1\texttt{,...,M}_n\ \texttt{ end}]\!]^{\texttt{Inst}}_{\texttt{D}} =$

```
type name_('self) = {
  (sup_('self) with)?
  __name: ~ℕ^Inst(name);
  ⟦M_1⟧^Inst; ...; ⟦M_n⟧^Inst
..}
type name = name_(name)
```

$[\![\texttt{class name ((<|<:) sup)}^?\ \texttt{M}_1\texttt{,...,M}_n\ \texttt{ end}]\!]^{\texttt{Class}}_{\texttt{D}} =$

```
type nameClass = {
  (supClass with)?
  __name: ~ℕ^Class(name);
  ⟦M_1⟧^Class; ...; ⟦M_n⟧^Class
..}
```

$$[\![\texttt{def } f \texttt{ : N}]\!]^{\texttt{Inst}}_{\texttt{M}} = f \texttt{ : } [\![\texttt{N}]\!]$$
$$[\![\texttt{@}x \texttt{ : T}]\!]^{\texttt{Inst}}_{\texttt{M}} = \texttt{\_\_attr\_}x \texttt{ : } [\![\texttt{T}]\!]$$
$$[\![\texttt{def self.}f \texttt{ : N}]\!]^{\texttt{Class}}_{\texttt{M}} = f \texttt{ : } [\![\texttt{N}]\!]$$
$$[\![\texttt{def initialize: F}_1 \texttt{ \% ... \% F}_n]\!]^{\texttt{Class}}_{\texttt{M}} = \texttt{new : } [\![\mathbb{I}(\texttt{F}_1) \texttt{ \% ... \% } \mathbb{I}(\texttt{F}_n)]\!]$$
$$[\![\texttt{def initialize: F}_1 \texttt{ \& ... \& F}_n]\!]^{\texttt{Class}}_{\texttt{M}} = \texttt{new : } [\![\mathbb{I}(\texttt{F}_1) \texttt{ \& ... \& } \mathbb{I}(\texttt{F}_n)]\!]$$
$$\mathbb{I}(\texttt{F}) = \text{dom}(\texttt{F}) \texttt{ -> self}$$

B. Contexte du stage

Mon stage s'est déroulé à l'IRIF, une unité mixte de recherche entre le CNRS et l'Université Paris-Cité. Pendant ce stage j'ai pu interagir régulièrement avec mon encadrant et d'autres chercheurs de l'IRIF. J'ai aussi pu interagir avec Yaozhu Sun, postdoc au National Institute of Informatics à Tokyo, notamment par des appels vidéos hebdomadaires.